Universidade Carioca

Trabalho Final - Lógica Nebulosa

Aprendizado de Máquina e Lógica Nebulosa

Q1 Classificação • Q2 Regressão • Q3 Aprendizado por Reforço

Aluno(a): _____

Professor: Prof. Mardônio França

28/10/2026

Visão geral do trabalho

- Questão 1 - Classificação: cachorro x não cachorro em imagens, com 3 modelos (KNN, Random Forest, SVM).
  - Saída DISCRETA (0 ou 1); a probabilidade de cada previsão é lida como grau de pertinência fuzzy.
- Questão 2 - Regressão: preço de passagens aéreas (Regressão Linear x Gradient Boosting).
  - Saída CONTÍNUA (R$); o valor já carrega a gradação que a lógica fuzzy formaliza.
- Questão 3 - Aprendizado por reforço: galinha atravessando uma rua com carros (Q-learning tabular).
  - Variação do labirinto: obstáculos móveis e sorteados; o agente aprende uma política.

» **Fio condutor: a realidade raramente é 0 ou 1. Os modelos produzem GRAUS (probabilidade, valor contínuo, valor Q) e a decisão crisp é uma etapa final que descarta parte dessa informação.**

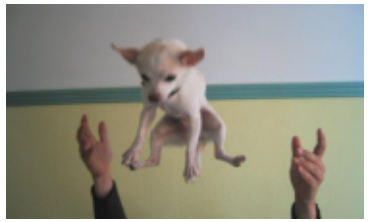
- Tudo em Python (scikit-learn, NumPy, pandas, matplotlib); código, dados, testes e README em cada pasta.

Q1 - Problema e dados

- Classificação binária: Cachorro (1, positivo) x Não cachorro (0).
- 1.000 imagens:
  - 500 cachorros: Stanford Dogs (120 raças).
  - 500 não cachorros: Animals-10 (gato, galinha, vaca, cavalo, ovelha; 100 de cada).
- Rótulo vem da pasta; 0 corrompidas, 0 duplicatas (SHA-256).
- Divisão estratificada por grupo (semente 42):
  - treino 640 | validação 160 | teste 200.
  - O teste é usado UMA vez, no final.

» **Limitação: só 5 espécies e duas bases diferentes (viés de fonte).**

cachorro



gato



galinha



vaca



cavalo



ovelha



Q1 - Representação das imagens e modelos

Modelo	Entrada	Como decide	Probabilidade
KNN (k=5)	pixels cinza 64×64 (4096)	5 vizinhos mais próximos votam	fração de votos (0; 0,2; ...)
Random Forest (100)	pixels cinza 64×64 (4096)	árvores de decisão votam	média das árvores
SVM RBF (C=1)	cor HSV (128) + HOG (1764)	fronteira de margem máxima	distância calibrada (Platt)

- Pixels crus mudam quando o animal se desloca na foto; o 3º modelo usa atributos mais estáveis:
  - Histograma HSV: QUAIS cores aparecem (pasto, céu, pelagem), não onde.
  - HOG (implementado em NumPy): direção das bordas em cada célula 16×16 → contornos e textura.
- StandardScaler dentro do Pipeline (ajustado só no treino); C do SVM escolhido na VALIDAÇÃO.
- Modelo final treinado com treino + validação (800 imagens) e avaliado uma vez no teste.

Q1 - Métricas no teste (200 imagens)

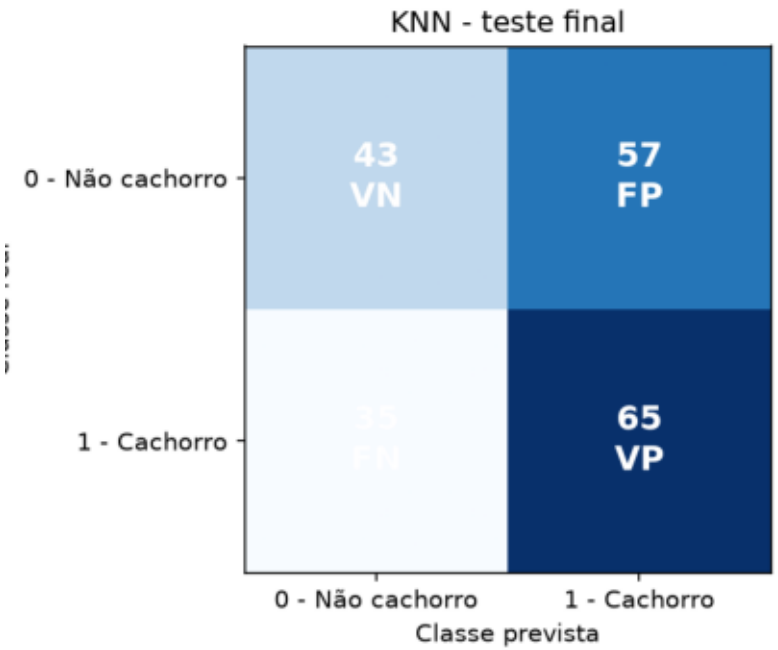
Modelo	Acurácia	Precisão	Recall	F1	F2	ROC-AUC	Acur. CV
KNN	0,540	0,533	**0,650**	0,586	0,623	0,570	0,554
Random Forest	0,620	0,618	0,630	**0,624**	**0,627**	0,653	0,625
SVM (cor + forma)	**0,640**	**0,659**	0,580	0,617	0,594	**0,730**	**0,681**

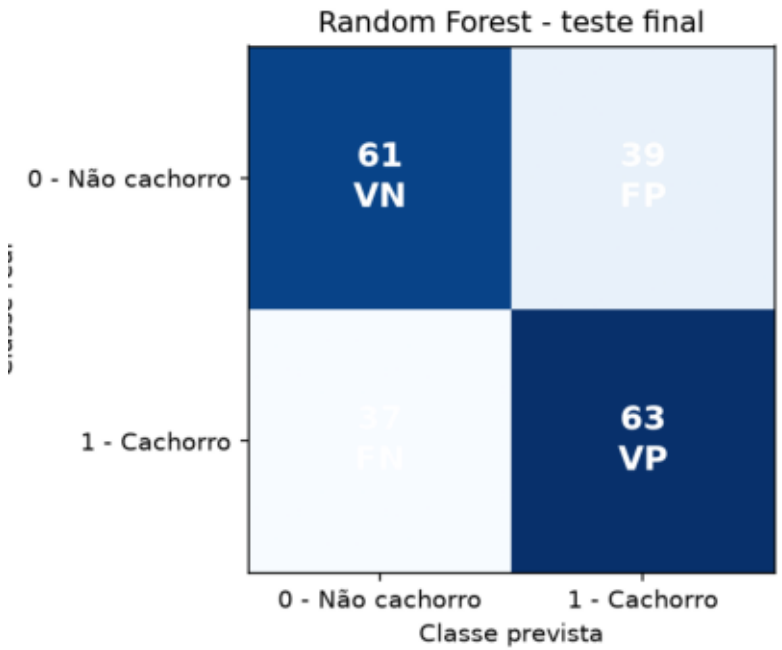
- Em vermelho, o melhor de cada métrica. F2 = (5·P·R)/(4·P + R): o recall pesa 4×. Acur. CV = validação cruzada.
- O SVM (cor + forma) lidera em acurácia, precisão, ROC-AUC e validação cruzada.
  - A ROC-AUC usa a probabilidade inteira (todos os limiares): mede quão bem o modelo ORDENA as imagens.
- O Random Forest tem o maior F1/F2 no limiar 0,5; o KNN tem o maior recall, mas muitos alarmes falsos.

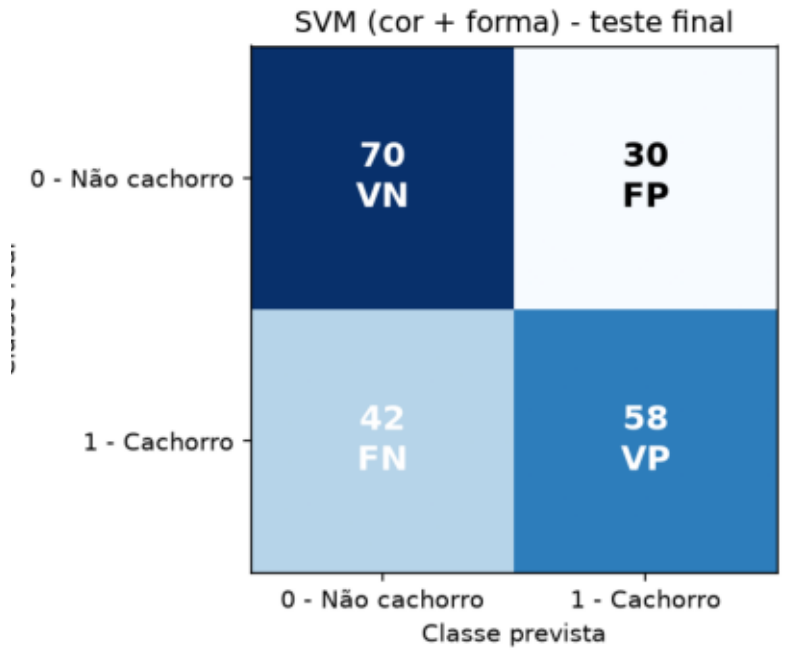
» **Com 200 imagens, a margem de erro da acurácia é ≈ ±7 pontos: diferenças pequenas não são conclusivas.**

- Desempenho modesto: representações simples, poucos dados; os três ficam acima do acaso (50%).

Q1 - Matrizes de confusão (teste)

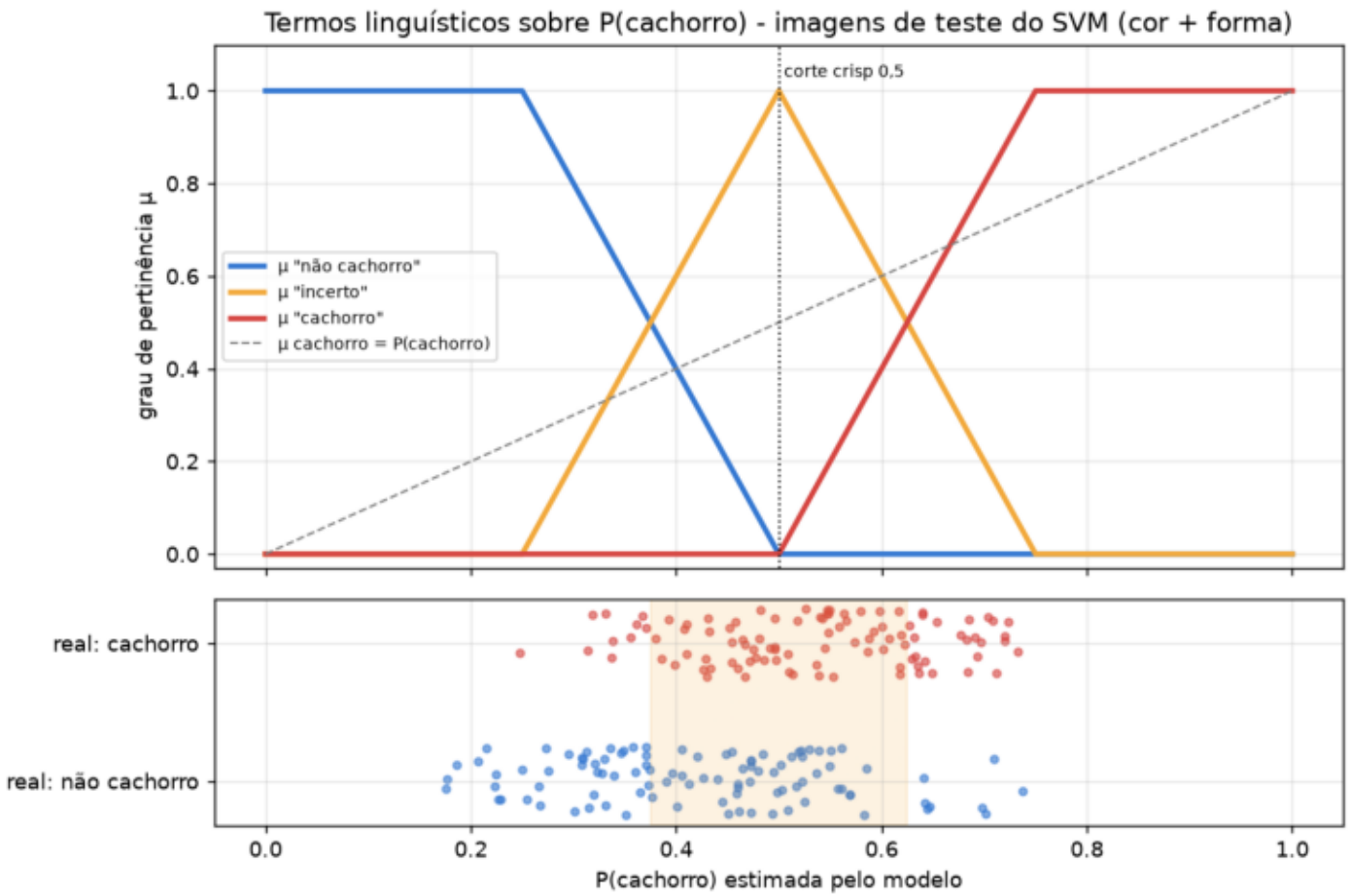






- Linhas = classe real; colunas = prevista. FP = animal tomado por cachorro; FN = cachorro perdido.
- KNN: muitos FP (chama quase tudo de cachorro). SVM: menos FP (mais preciso), porém mais FN.

Q1 c) Probabilidade da predição = grau de pertinência



- predict_proba(X) devolve P(cachorro) para cada imagem.
- Lógica clássica: a imagem É ou NÃO É cachorro (0 ou 1).
- Lógica fuzzy: μ_cachorro = P(cachorro) e μ_não = 1 − P.
  - P = 0,74 → 'bastante cachorro'; P = 0,52 → quase totalmente ambíguo.
- Três termos linguísticos sobre P(cachorro): 'não cachorro', 'incerto', 'cachorro' (somam 1).

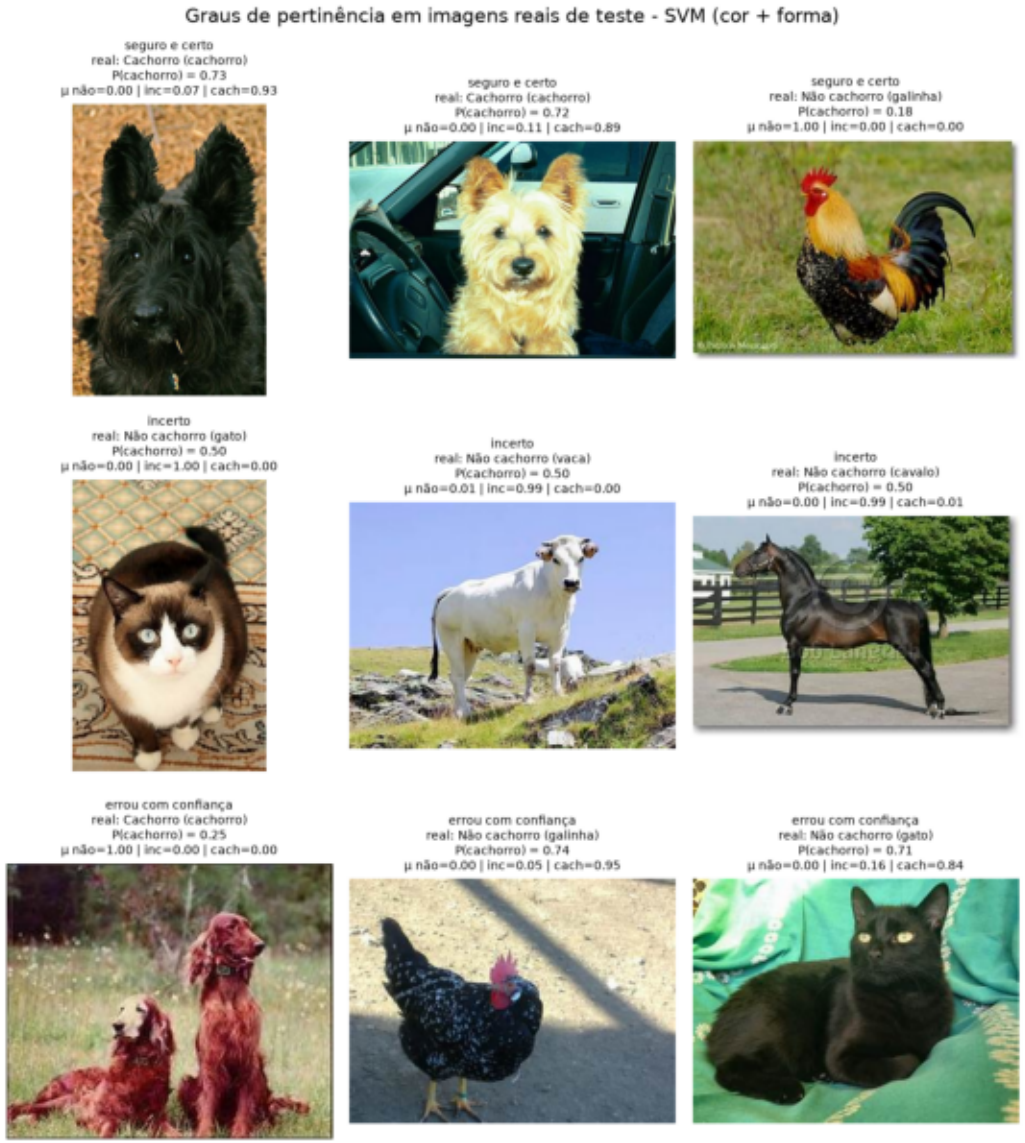
» **predict (corte em 0,5) é uma DEFUZZIFICAÇÃO: transforma o grau em sim/não e joga fora a incerteza.**

Q1 c) A incerteza diz quando confiar na resposta

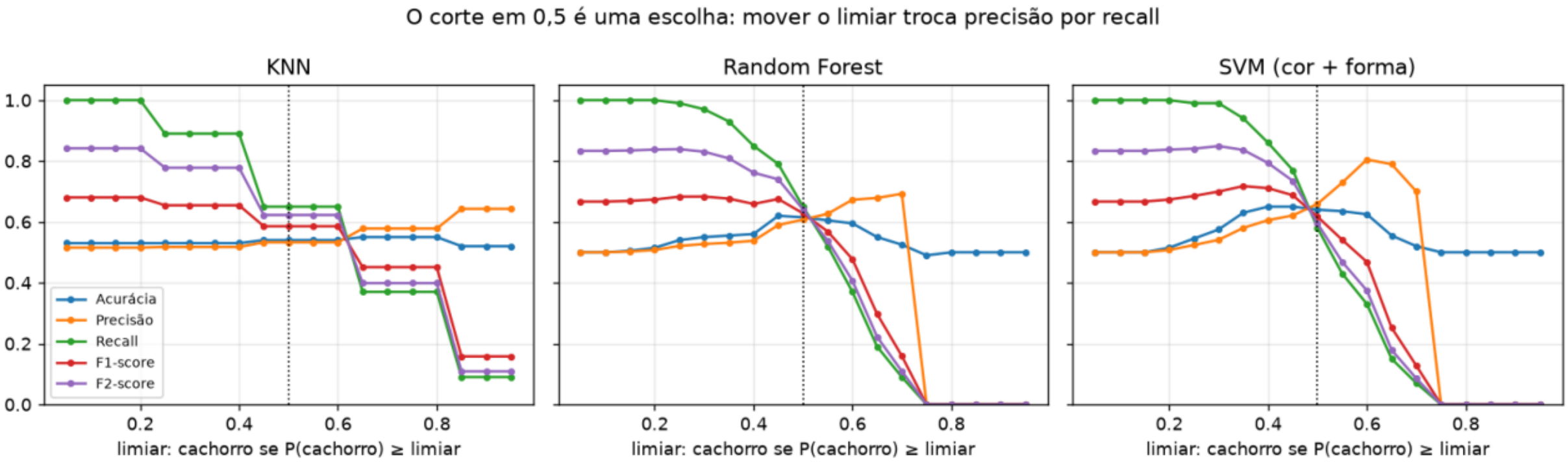
Modelo	'não cachorro'	'incerto'	'cachorro'
KNN	60,7% (28)	50,0% (108)	57,8% (64)
Random Forest	65,7% (35)	59,4% (133)	68,8% (32)
SVM (cor + forma)	80,4% (51)	53,4% (116)	75,8% (33)

acerto da decisão crisp (nº de imagens) por termo dominante

- SVM: pertinência clara → 80% e 76% de acerto;
  - zona incerta → 53% (quase cara ou coroa).
- O grau de pertinência mede a confiança; a saída crisp não.
- Um sistema fuzzy poderia responder 'não sei → revisar' na zona incerta.
- KNN: só 6 valores de P (pertinência em degraus); SVM: 200 valores distintos.



Q1 c) O limiar 0,5 é uma escolha (fronteira crisp)



- Mover o corte troca precisão por recall: baixar o limiar encontra mais cachorros (F2 ↑), com mais alarmes falsos.
- A fronteira entre 'cachorro' e 'não cachorro' no espaço de atributos não é nítida: o próprio conceito é difuso
  - (alguns gatos e cães pequenos se parecem mais entre si do que um chihuahua com um são-bernardo).

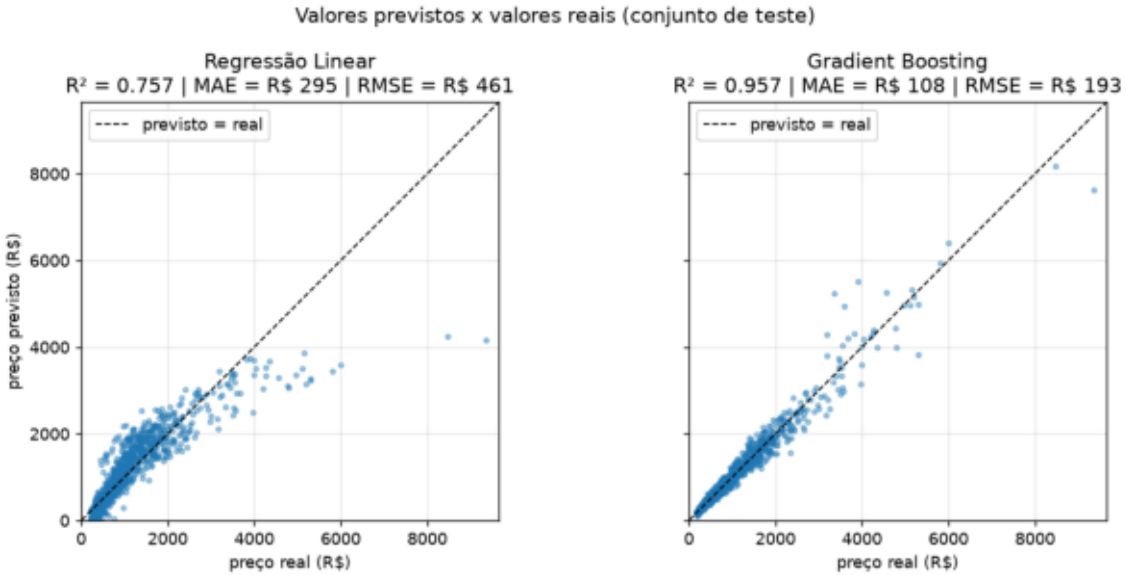
Q2 - Problema, dados e modelos

- Prever o PREÇO (R$) de uma passagem aérea: variável contínua.
- 5.000 passagens SINTÉTICAS (gerar_dados.py): regra conhecida + ruído de ≈10%.
  - Não há base pública offline; a regra conhecida permite checar o que o modelo aprendeu.
  - Variáveis: distância, antecedência, escalas, alta temporada, companhia, classe, dia da semana, período.
- preço = (90 + 0,38·km) × companhia × classe × (1 + 1,2·e^(−dias/12)) × escalas × dia × temporada × período × ruído
- a) Divisão treino 80% (4.000) / teste 20% (1.000), semente 42.
- Pré-processamento no Pipeline: StandardScaler (numéricas) + OneHotEncoder (categóricas).
- Modelos:
  - Regressão Linear: soma de pesos × atributos (referência).
  - Gradient Boosting: 400 árvores pequenas, cada uma corrige o erro das anteriores (não linear, interações).

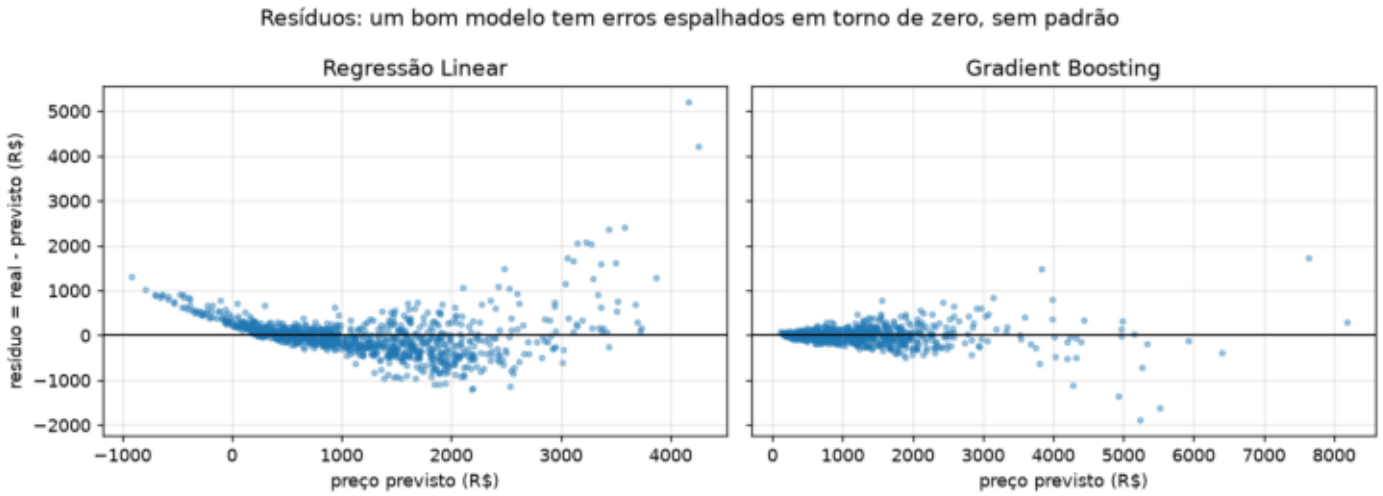
Q2 b) Métricas no teste e previsto x real

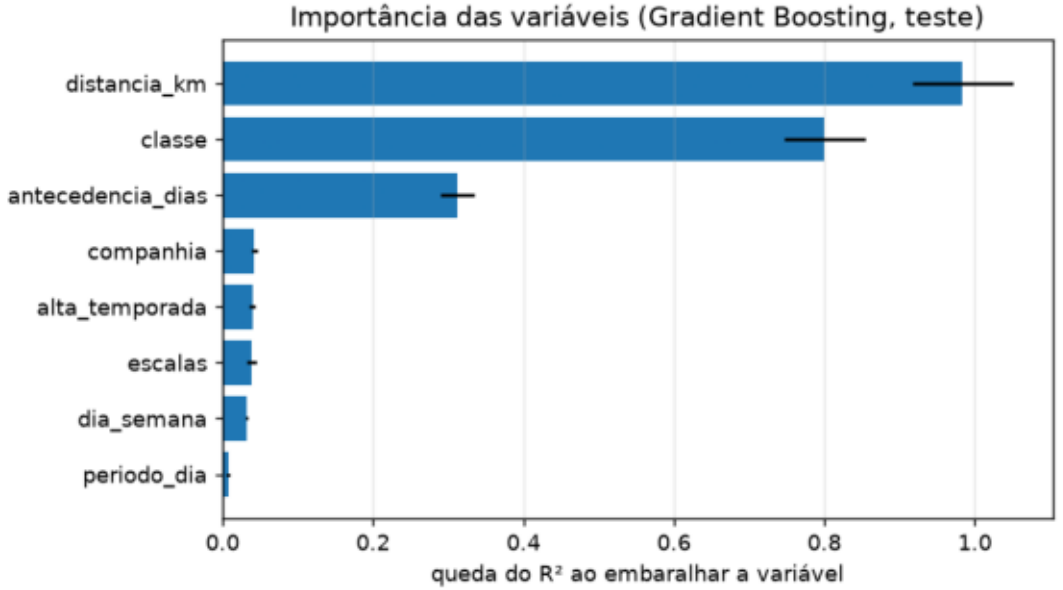
Modelo	MAE (R$)	MSE (R$²)	RMSE (R$)	R²
Regressão Linear	294,73	212.281	460,74	0,757
Gradient Boosting	107,66	37.134	192,70	0,957

- MAE: erro médio absoluto → o Boosting erra ≈ R$ 108 por passagem.
- MSE: média dos erros²; pune erros grandes (unidade R$²).
- RMSE = √MSE, de volta em R$ (R$ 193).
- R²: 95,7% da variação do preço explicada; o resto ≈ ruído dos dados.
- A Linear perde porque a regra é multiplicativa e não linear (curvatura no gráfico).



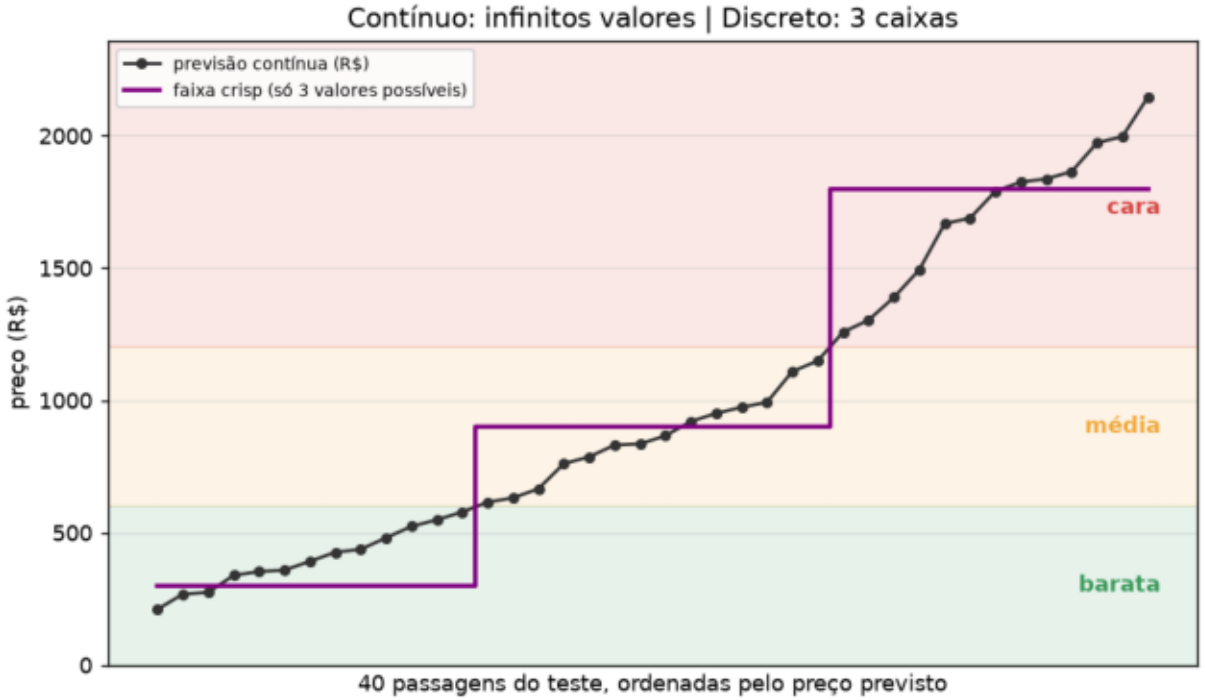
Q2 b) Resíduos e importância das variáveis

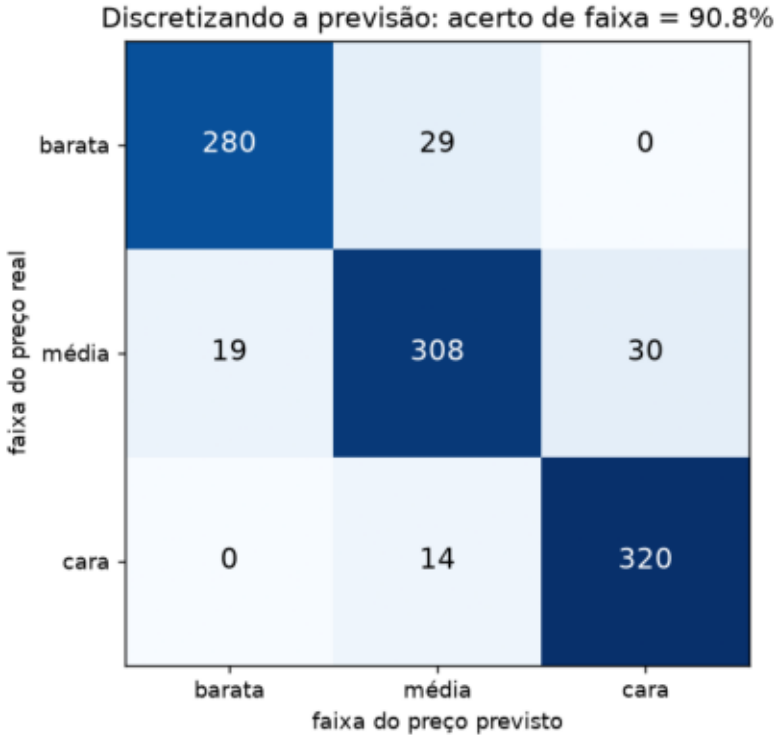




- Resíduos do Boosting em torno de zero, sem padrão. A Linear mostra curvatura: falta não linearidade.
- Distância, classe e antecedência são as mais importantes, exatamente como na regra que gerou os dados.

Q2 c) Resultado contínuo x classificação discreta





- Q1: saída discreta, 1 entre 2 rótulos. Q2: saída contínua, um número real (996 valores distintos em 1000 passagens).
- Discretizando em faixas (barata < R$ 600 ≤ média < R$ 1.200 ≤ cara), a regressão vira classificação de 3 classes.
  - Perde-se informação: R$ 610 e R$ 1.150 viram a mesma 'média'; R$ 590 e R$ 609 caem em classes diferentes.
  - Os erros de faixa aparecem só entre faixas vizinhas, justamente nas fronteiras arbitrárias.

Q2 d) A lógica nebulosa já está no resultado contínuo



Previsto	Faixa crisp	μ barata	μ média	μ cara
R$ 450	barata	0,75	0,25	0,00
R$ 590	barata	0,52	0,48	0,00
R$ 609	média	0,48	0,52	0,00
R$ 903	média	0,00	0,99	0,01
R$ 1.151	média	0,00	0,58	0,42
R$ 1.249	cara	0,00	0,42	0,58
R$ 1.997	cara	0,00	0,00	1,00

- O valor contínuo carrega GRAU: R$ 1.150 é 'um pouco média (0,58) e um pouco cara (0,42)', não uma coisa ou outra.
- Crisp: R$ 590 é barata e R$ 609 é média. Fuzzy: ambas ≈ 0,5/0,5, praticamente iguais (a leitura humana correta).

» **A regressão é o problema 'não defuzzificado': na Q1 o corte 0,5 transforma o grau em rótulo; aqui o grau é a saída.**

- Há ainda a incerteza do próprio número: RMSE ≈ R$ 193 → 'R$ 1.150' é, na prática, algo entre ~R$ 960 e ~R$ 1.340.

Q3 a) Ambiente: galinha atravessando a rua

- Grade 11 × 8: linha 0 = calçada (início), 10 = grama (chegada), 1 a 9 = pistas.
  - 2 carros por pista; pistas alternam o sentido; carros sorteados a cada episódio.
- Estado: (linha, coluna, frente_ocupada, frente_vindo, aqui_vindo).
  - 11 × 8 × 2 × 2 × 2 = 704 estados (em vez de ~10^{13} com o mapa inteiro).
- Ações: CIMA, BAIXO, ESQUERDA, DIREITA, PARADA.
- Fim do episódio: atropelada, chegou ou 100 passos.
- É um 'labirinto' com paredes móveis: não existe caminho fixo, e o agente aprende uma POLÍTICA.

Evento	Recompensa
cada passo	−1
linha mais alta que todas as anteriores	+5
atropelada (fim)	−50
chegou do outro lado (fim)	+100

galinha_env.py (só as regras, sem aprendizado)

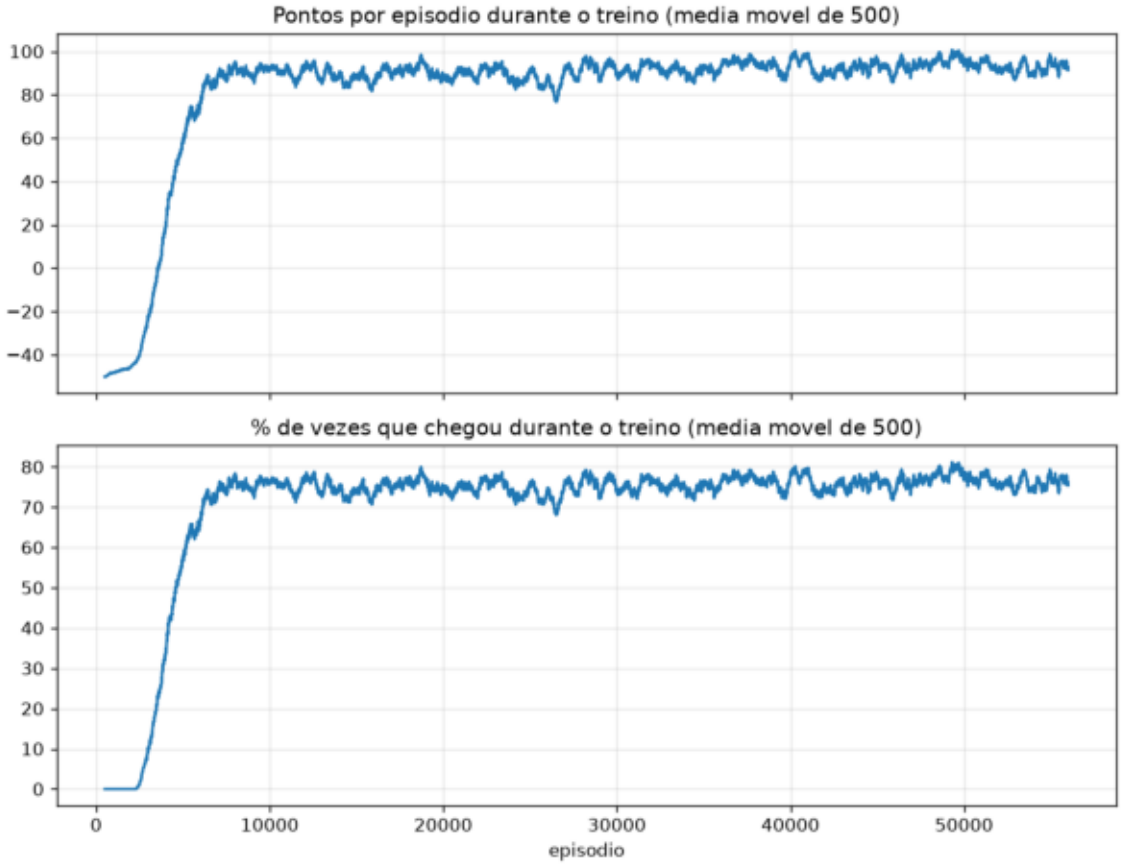
Q3 a) Agente: Q-learning tabular

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \,]$$

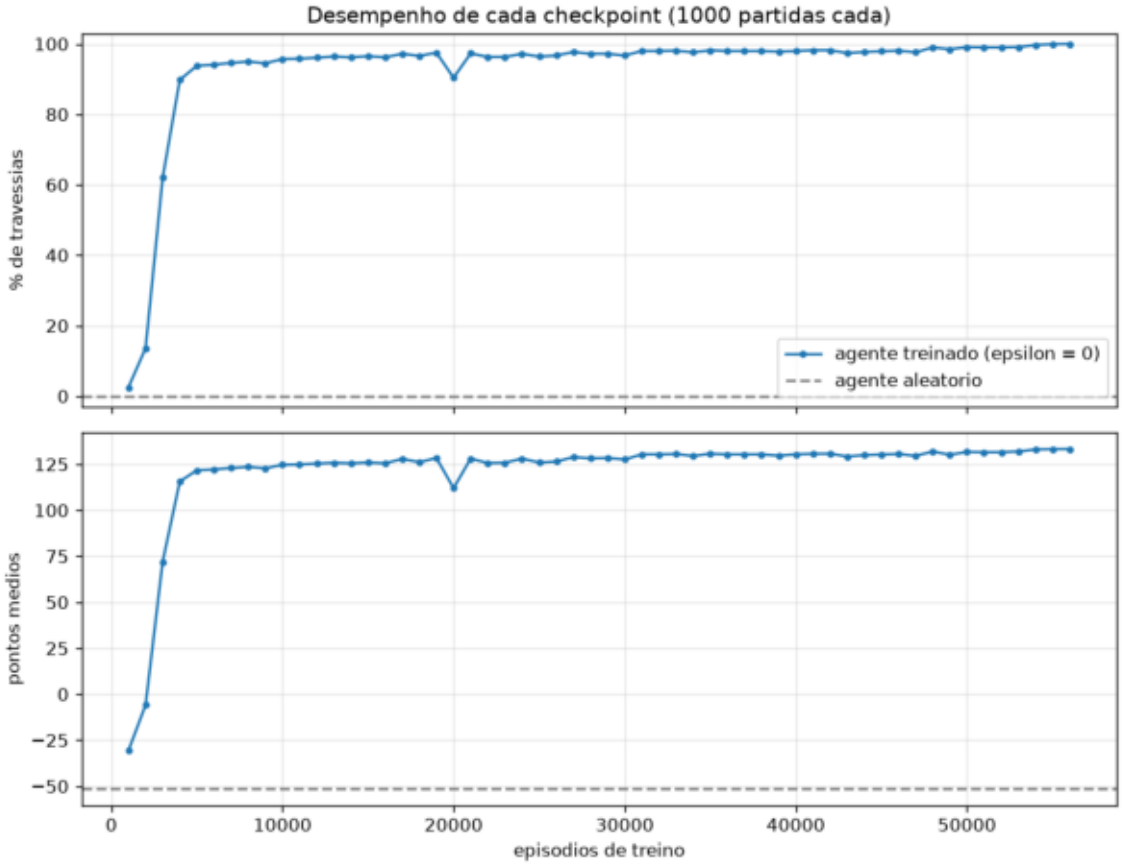
- Tabela Q: estado → 5 valores (um por ação) = quanto vale cada ação naquele estado.
- α = 0,1 (taxa de aprendizado): corrige 10% do erro a cada passo.
- γ = 0,95 (desconto): recompensas futuras valem um pouco menos que as imediatas.
- Epsilon-greedy: com chance ε faz ação aleatória (explora); senão, a de maior Q (usa o que aprendeu).
  - ε começa em 1,0 e decai ×0,9995 por episódio até 0,05 (≈ episódio 6.000).
- Implementado do zero, sem bibliotecas de RL (treinar.py: escolher_acao, aprender, rodar_episodio).

Q3 b) Evolução do aprendizado ao longo dos episódios

durante o treino (com exploração ε)

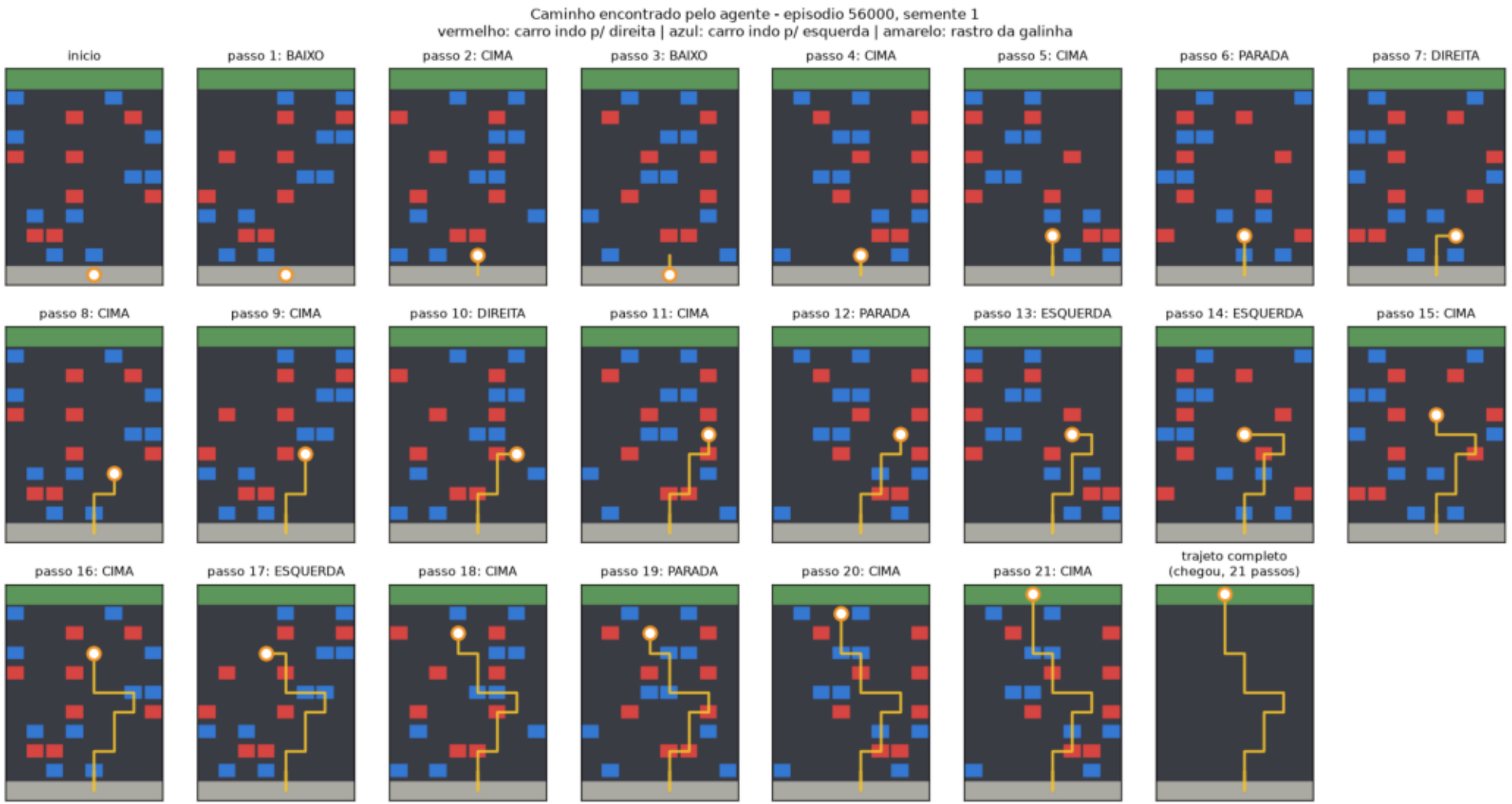


cada checkpoint SEM exploração, 1.000 partidas



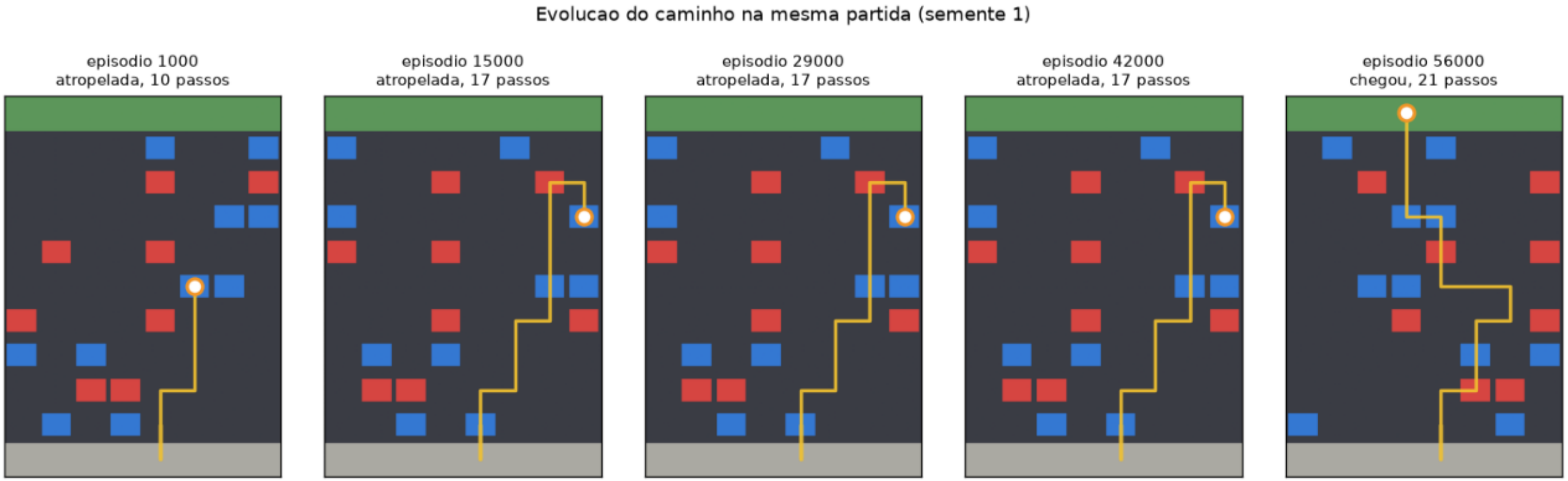
- Travessias sem exploração - aleatório: 0,0% | ep. 1.000: 2,5% | ep. 3.000: 62,0% | ep. 5.000: 93,8% | ep. 56.000: 100,0%
- No treino a curva fica em ≈76% porque 5% das ações continuam aleatórias (ε mínimo).

Q3 b) Caminho final encontrado pelo agente



- Resultado: chegou em 21 passos | pontos: 129. Recua (BAIXO), espera (PARADA) e desvia: ninguém programou isso.

Q3 b) A mesma partida jogada por épocas diferentes



- Mesmos carros (mesma semente); só muda a tabela Q carregada de cada checkpoint.
- Início: sobe direto e é atropelada. Meio do treino: vai mais longe, mas cai na mesma armadilha. Final: atravessa.
- Relação com o fuzzy: os valores Q são graus contínuos de 'quão boa' é cada ação; o argmax é a decisão crisp.

Conclusões

- Q1: três classificadores comparados por matriz de confusão, acurácia, precisão, recall, F1, F2 e ROC-AUC.
  - O SVM com cor + HOG foi o melhor no geral, mas com resultados modestos; a incerteza das previsões é alta.
  - A probabilidade é um grau de pertinência: os erros se concentram onde ela é ambígua.
- Q2: Gradient Boosting com $R^2 \approx 0{,}96$ no teste, contra 0,76 da Regressão Linear.
  - O preço contínuo já é graduado; discretizar em faixas cria fronteiras artificiais.
  - Funções de pertinência expressam 'um pouco média e um pouco cara'.
- Q3: Q-learning aprendeu a atravessar em 100% das partidas de avaliação (contra 0% do aleatório).

» **Lógica nebulosa: classificação, regressão e reforço produzem GRAUS; a decisão crisp é só o último passo.**

» **Manter o grau permite medir a confiança, ajustar limiares e responder 'não sei' quando for o caso.**

Repositório e como executar

- Repositório: (link do GitHub)
- pip install -r requirements.txt (Python 3.10+)
- Q1: cd questao1_classificacao → python preparar_dados.py → python treinar.py → python analise_fuzzy.py
  - Interface: streamlit run app.py
- Q2: cd questao2_regressao → python gerar_dados.py → python regressao.py
- Q3: cd questao3_reforco_galinha → python avaliar.py | python visualizador.py (pygame)
  - Treinar do zero: python treinar.py --do-zero --episodios 20000 --checkpoint-cada 1000
- Testes: python -m pytest em cada pasta (69 testes no total).
- Esta apresentação: python gerar_apresentacao.py